Light

GCSE Computer Science - Programming Testing and Robustness (Set 2)

Over **1,100,000+ marks** of exam-aligned content across **67 qualifications**. 4,200+ auto-marked assessments & long-answer practice. No login required.

GCSE • AQA 8525 • Grades 3—9 | Topic test | 30 min | 55 marks | Answered: 0 / 30

00:00 Start Pause Reset

← Back to Programming Testing And Robustness

Multiple-choice topic test for **GCSE Computer Science (AQA 8525)**. You may skip questions. When you submit, you'll see your score, a grade estimate, and worked explanations. **Tip:** use the stopwatch above to time yourself.

Q1 (1 mark)

Why do programmers include data validation in their programs?

- ○ **A.** To make programs run faster
- ○ **B.** To prevent invalid or unreasonable data from being processed
- ○ **C.** To make the code shorter
- ○ **D.** To change the programming language

Q2 (1 mark)

What is the purpose of using test data when testing a program?

- ○ **A.** To make the program more colourful
- ○ **B.** To encrypt the program's source code
- ○ **C.** To verify that the program produces correct outputs for given inputs
- ○ **D.** To delete errors automatically

Q3 (1 mark)

Which statement best describes when a syntax error is detected?

- ○ **A.** Only when the program finishes running
- ○ **B.** When the program produces incorrect output
- ○ **C.** After the user enters invalid data
- ○ **D.** Before or when the program attempts to run, preventing execution

Q4 (1 mark)

A program runs without error messages but calculates an employee's pay incorrectly. What type of error is this?

- ○ **A.** Syntax error
- ○ **B.** Compilation error
- ○ **C.** Logic error
- ○ **D.** Hardware error

Q5 (1 mark)

Why is boundary testing particularly important?

- ○ **A.** Because many programming errors occur at the edges of valid ranges due to incorrect comparison operators
- ○ **B.** Because boundaries are always where programs crash
- ○ **C.** Because boundary values are easier to remember
- ○ **D.** Because users only ever enter boundary values

Q6 (1 mark)

What should happen when a well-designed program receives erroneous test data?

- ○ **A.** The program should crash immediately
- ○ **B.** The program should accept the data without question
- ○ **C.** The program should delete itself
- ○ **D.** The program should reject the data and provide appropriate feedback

Q7 (1 mark)

A simple authentication system typically requires which two pieces of information?

- ○ **A.** Username and email address
- ○ **B.** Password and PIN

- ○ **C.** Username and password
- ○ **D.** First name and last name

Q8 (1 mark)

A cinema booking system needs to validate the number of tickets (maximum 10 per booking). Which validation type is most appropriate?

- ○ **A.** Range check
- ○ **B.** Format check
- ○ **C.** Lookup check
- ○ **D.** Check digit

Q9 (1 mark)

A username must be between 3 and 15 characters. Which validation check is needed?

- ○ **A.** Length check
- ○ **B.** Presence check
- ○ **C.** Type check
- ○ **D.** Range check

Q10 (1 mark)

A program accepts temperatures from -20 to 50 degrees Celsius. Which value represents normal test data?

- ○ **A.** -21
- ○ **B.** 51
- ○ **C.** "warm"
- ○ **D.** 15

Q11 (1 mark)

Which of the following is most likely to be a syntax error?

- ○ **A.** Using the wrong formula to calculate tax
- ○ **B.** A loop that runs one time too many
- ○ **C.** A missing closing bracket in a print statement
- ○ **D.** Dividing by zero during calculation

Q12 (1 mark)

What is the relationship between testing and debugging?

- ○ **A.** They are the same thing
- ○ **B.** Testing finds errors; debugging fixes them
- ○ **C.** Debugging finds errors; testing fixes them
- ○ **D.** Neither involves finding errors

Q13 (2 marks)

What is output when the user enters 25?

```
mark = INPUT("Enter mark: ")
IF mark >= 0 AND mark <= 20 THEN
    OUTPUT "Valid mark"
ELSE
    OUTPUT "Mark out of range"
ENDIF
```

- ○ **A.** Valid mark
- ○ **B.** 25
- ○ **C.** Error
- ○ **D.** Mark out of range

Q14 (2 marks)

Identify the type of error in this code:

```
FOR i = 1 TO 10
    OUTPUT i
ENDFOR
OUTPUT "Done"
```

- ○ **A.** Logic error - the loop counts wrong
- ○ **B.** No error - the code is correct
- ○ **C.** Syntax error - FOR should use <- not =
- ○ **D.** Runtime error - infinite loop

Q15 (2 marks)

This code should find the sum of numbers 1 to 5 (which equals 15). What does it actually output?

```
total = 0
FOR i <- 1 TO 5
    total = i
ENDFOR
OUTPUT total
```

- ○ **A.** 5
- ○ **B.** 15
- ○ **C.** 0
- ○ **D.** 1

Q16 (2 marks)

What is output when the user enters username "user1" and password "pass123"?

```
correctUser = "admin"
correctPass = "pass123"

enteredUser = INPUT("Username: ")
enteredPass = INPUT("Password: ")

IF enteredUser = correctUser OR enteredPass = correctPass THEN
    OUTPUT "Access granted"
ELSE
    OUTPUT "Access denied"
ENDIF
```

- ○ **A.** Access denied

- B. Access granted
- C. user1
- D. Error

Q17 (2 marks)

A program accepts ages from 16 to 65 for a job application. Which set contains appropriate boundary test data?

- A. 20, 30, 40, 50
- B. 15, 16, 65, 66
- C. 16, 65
- D. 0, 100

Q18 (2 marks)

What is output when the user enters just spaces (" ")?

```
email = INPUT("Enter email: ")
IF email = "" THEN
    OUTPUT "Email required"
ELSE
    OUTPUT "Email received"
ENDIF
```

- A. Email required
- B. Error
- C.

- ○ **D.** Email received

Q19 (2 marks)

This code should check if someone is eligible to vote (18 or older). Identify the error:

```
age = INPUT("Enter your age: ")
IF age > 18 THEN
    OUTPUT "You can vote"
ELSE
    OUTPUT "You cannot vote"
ENDIF
```

- ○ **A.** Syntax error - missing ENDIF
- ○ **B.** Logic error - should be >= 18, not > 18, as 18-year-olds can vote
- ○ **C.** No error - the code is correct
- ○ **D.** Runtime error - age might not be a number

Q20 (2 marks)

What is output when "OpenSesame" (10 characters) is entered?

```
password = INPUT("Enter password: ")
IF LENGTH(password) >= 6 AND LENGTH(password) <= 12 THEN
    OUTPUT "Password length OK"
ELSE
    OUTPUT "Password must be 6-12 characters"
ENDIF
```

- ○ **A.** Password length OK

- B. Password must be 6-12 characters
- C. 10
- D. OpenSesame

Q21 (2 marks)

A program calculates postage costs based on parcel weight in kilograms (positive numbers only). Which value is erroneous test data?

- A. 2.5
- B. 0.1
- C. 10
- D. -3

Q22 (2 marks)

Identify and categorise the error in this code:

```
price = 100
discount = 0.15
final = price - price  discount
OUTPUT final
```

- A. Syntax error - missing operator between price and discount
- B. Logic error - wrong calculation formula
- C. No error - code is correct

- ○ **D.** Runtime error - division by zero

Q23 (2 marks)

A robust email validation should check multiple things. Which combination is most appropriate?

- ○ **A.** Only check the field is not empty
- ○ **B.** Check it is not empty, contains @ symbol, and has text before and after the @
- ○ **C.** Only check it contains letters
- ○ **D.** Only check the password matches

Q24 (3 marks)

Analyse this login system. What happens if the user enters "admin" and "letmein" on the first attempt?

```
storedUser = "admin"
storedPass = "secure99"
attempts = 0
success = False

WHILE attempts < 3 AND success = False
    user = INPUT("Username: ")
    pass = INPUT("Password: ")
    attempts = attempts + 1

    IF user = storedUser AND pass = storedPass THEN
        success = True
        OUTPUT "Login successful"
    ELSE
        OUTPUT "Login failed - " + (3 - attempts) + " attempts re
    ENDIF
ENDWHILE
```

```
IF success = False THEN
    OUTPUT "Account locked"
ENDIF
```

- ○ **A.** Login successful
- ○ **B.** Login failed - 2 attempts remaining, then the loop continues
- ○ **C.** Account locked immediately
- ○ **D.** Login failed - 3 attempts remaining

Q25 (3 marks)

This validation routine checks a student's mark. What is output for input "85"?

```
mark = INPUT("Enter mark: ")
valid = True
message = ""

IF mark = "" THEN
    valid = False
    message = "Mark cannot be empty"
ELSE IF mark < 0 THEN
    valid = False
    message = "Mark cannot be negative"
ELSE IF mark > 100 THEN
    valid = False
    message = "Mark cannot exceed 100"
ENDIF

IF valid = True THEN
    OUTPUT "Mark accepted: " + mark
ELSE
    OUTPUT message
ENDIF
```

- ○ **A.** Mark cannot be empty
- ○ **B.** Mark cannot exceed 100
- ○ **C.** Mark accepted: 85
- ○ **D.** Mark cannot be negative

Q26 (3 marks)

This code should calculate the average of positive numbers only, but has a bug. With input [5, -2, 10, -3, 15], it outputs 5 instead of 10. Identify the error.

```
numbers = [5, -2, 10, -3, 15]
total = 0
count = 0

FOR i <- 0 TO 4
    IF numbers[i] > 0 THEN
        total = total + numbers[i]
    ENDIF
    count = count + 1
ENDFOR

average = total / count
OUTPUT average
```

- ○ **A.** The IF condition should be >= 0
- ○ **B.** The count increment is outside the IF block, counting all numbers instead of just positive ones
- ○ **C.** The loop should start at 1
- ○ **D.** Division should use integer division

Q27 (3 marks)

A program validates student ages for a school system (ages 4-18 inclusive). Design comprehensive test data.

- ○ **A.** Normal: 10, Boundary: 3, 4, 18, 19, Erroneous: -5, "twelve"
- ○ **B.** Test only 4 and 18
- ○ **C.** Test 1, 2, 3, 4, 5, 6, 7, 8, 9, 10
- ○ **D.** Normal: 4, 18, Boundary: 10, Erroneous: 5

Q28 (3 marks)

How many errors does this code contain, and what types are they?

```
1: num = INPUT("Enter number: ")
2: IF num > 10
3:     result = num * 2
4: ELSE
5:     result = num / 0
6: ENDIF
7: OUTPUT result
```

- ○ **A.** One syntax error only (missing THEN)
- ○ **B.** One logic error only (division by zero)
- ○ **C.** No errors - the code is correct
- ○ **D.** Two errors: syntax (missing THEN) and logic (division by zero)

Q29 (3 marks)

A bank ATM validates withdrawal amounts (multiples of 10, between 10 and 300). Analyse which validation is most robust.

- ○ **A.** IF amount >= 10 AND amount <= 300 THEN valid
- ○ **B.** IF amount MOD 10 = 0 THEN valid
- ○ **C.** IF amount >= 10 AND amount <= 300 AND amount MOD 10 = 0 THEN valid
- ○ **D.** IF amount = 10 OR amount = 20 OR amount = 50 THEN valid

Q30 (3 marks)

A student wrote code to check if a number is prime. Testing with input 9 outputs "Prime" when it should output "Not prime" (9 = 3 * 3). Identify the bug.

```
num = INPUT("Enter number: ")
isPrime = True

FOR i <- 2 TO num - 1
    IF num MOD i = 0 THEN
        isPrime = False
    ENDIF
ENDFOR

IF isPrime = True THEN
    OUTPUT "Prime"
ELSE
    OUTPUT "Not prime"
ENDIF
```

- ○ **A.** The MOD operation is wrong - should check num / i = 0
- ○ **B.** The loop should start at 1, not 2
- ○ **C.** The code logic is correct; the expected output for 9 should actually be Prime

○ **D.** The loop iterates TO num - 1, so for num = 9 it only checks 2 to 8, missing the divisor 3 - wait, 3 IS checked, so the logic seems correct

Total: **55** marks • Time: **30 min**

↑ Back to top

Submit answers

Williams Physics Education — over **1,100,000+ marks** of exam-aligned content and **4,200+ auto-marked assessments** across 67 qualifications at GCSE, A-Level, AP & Professional Qualifications. Free to use, no login required.

About & Contact • Privacy • Terms • Accessibility • Copyright • DMCA • YouTube

WPE is free for everyone. If our resources help you or your students, you can support the project here.

© 2026 Williams Physics Education. All rights reserved.